Computação em Nuvem

Trabalho A1

Thiago Pinheiro de Araújo
EMAp/FGV
2026.1

Objetivo

Projetar, implementar e implantar a plataforma **DijkFood**, um serviço de delivery de comida capaz de calcular rotas sobre o grafo viário de São Paulo, registrar o ciclo de vida completo de cada pedido e atender aos requisitos disponibilidade, escalabilidade automática e otimização de custo.

Trabalho

Contexto

- A **DijkFood** conecta clientes, restaurantes e entregadores.
- Clientes realizam pedidos, a plataforma calcula a rota mais curta sobre o grafo viário de São Paulo
  - Utilizando dados da OpenStreetMap através da osmnx.
- Entregadores aceitam pedidos, executam entregas e reportam sua posição e o estado do pedido ao longo do ciclo de vida.
- Recursos AWS disponíveis para construir a solução:
  - EC2, RDS, DynamoDB, ECS, S3.

Trabalho

Requisitos funcionais

- Um administrador deverá ser capaz de cadastrar um usuário informando o seu nome, e-mail, telefone e localização geográfica da sua residência.

- Um administrador deverá ser capaz de cadastrar um restaurante informando o seu nome, tipo de cozinha, e localização geográfica da sua loja.

- Um administrador deverá ser capaz de cadastrar um entregador informando o seu nome, tipo de veículo e localização geográfica inicial
  - Após realizar uma entrega o courier espera por outra no mesmo local, sem deslocamento.
  - Um courier não pode ser escolhido para uma entrega caso esteja realizando outra.

Trabalho

Requisitos funcionais

- Um cliente deverá ser capaz de criar um pedido informando o restaurante e a lista de itens que deseja
  - Ao receber o pedido o sistema deverá encontrar o entregador mais adequado para realizar a entrega e calcular a rota.

- Durante o ciclo de vida de uma entrega o seu estado deverá ser constantemente notificado
  - Status válidos, em ordem: *CONFIRMED* → *PREPARING* → *READY_FOR_PICKUP* → *PICKED_UP* → *IN_TRANSIT* → *DELIVERED*.
  - Transições fora dessa sequência devem ser rejeitadas.

Trabalho

Requisitos funcionais

- Um entregador deverá reportar sua posição continuamente
  - Desde o deslocamento para pegar o pedido no restaurante, até o endereço do cliente.
- Um cliente deverá ser capaz de consultar o estado corrente de um pedido, informando o seu estado, o entregador selecionado e a sua posição.
- Um administrador deverá ser capaz de consultar o histórico de pedidos de um cliente (retornado em ordem cronológica).
- Um administrador deverá ser capaz de consultar o histórico de eventos de um pedido (retornado em ordem cronológica).

Trabalho

Interface de acesso

- O serviço deverá ser fornecido através de uma **API REST**.
  - Todas as entidades deverão suportar as operações esperadas pelo seu ciclo de vida (criação, consulta, atualização de estado).
- A criação de um pedido deverá garantir que os respectivos cliente e restaurante existam
  - A operação deverá ser atômica: nenhum estado parcial deve ser observável em caso de falha.
- O registro de um evento de entrega deve validar a transição de status antes de persistir.

Trabalho

Requisitos não-funcionais

- Volumes de operação esperados:

Cenário	Pedidos/segundo
Operação normal	10
Pico (almoço/jantar)	50
Evento especial	200

- Entregadores em rota reportam posição a cada 100ms.
- Considere que existem 3x mais entregadores que clientes.

Trabalho

Requisitos não-funcionais

- O sistema deverá garantir que picos de requisições em qualquer componente não degradem a responsividade dos demais componentes
  - A definição dos componentes faz parte do trabalho.
- Cada componente do sistema deverá ser capaz de escalar horizontalmente de forma independente, de acordo com a demanda específica do seu *workload*.
- O serviço deve permanecer disponível na falha de qualquer instância individual de computação, sem interrupção perceptível para o cliente.
- A API deve responder em menos de 500ms no percentil 95 para operações de consulta e registro.
- Dados confirmados devem ser duráveis e tolerantes à falha de uma zona de disponibilidade.

Trabalho

Simulador de carga

- O grupo deve implementar um **simulador de carga** para validar os requisitos não-funcionais. O simulador deve:
  - Popular o sistema com usuários, restaurantes e entregadores através da API.
  - Emitir pedidos em uma taxa configurável (pedidos/segundo) simulando os três cenários de carga descritos
    - Para cada pedido, registrar os eventos de entrega e as atualizações de posição do entregador durante a entrega.
  - Coletar e reportar a latência de cada chamada à API e confirmar que o throughput efetivo atende à taxa configurada.
- O resultado deverá demonstrar que a saturação de um componente não se propaga aos demais
  - Exemplo: um pico de cálculos de rota não deve aumentar a latência de consultas de status.

Trabalho

Implantação e teste de carga

- O grupo deve entregar um script Python (**deploy.py**) capaz de, sem qualquer intervenção manual:
  - Criar todos os recursos AWS necessários para a operação do sistema.
  - Realizar o deploy de todos os serviços.
  - Executar o experimento de carga com o simulador.
  - Destruir todos os recursos ao final da execução.
- Observações**:**
  - Qualquer erro durante a execução zera o critério correspondente na avaliação.
  - Credenciais AWS não devem constar no repositório
    - Utilizar *~/.aws/credentials*.

Trabalho

Entregáveis

- O grupo deverá apresentar um relatório em PDF (máximo 10 páginas) contendo:
  - **Diagrama de arquitetura contendo** todos os recursos AWS utilizados e o fluxo de dados entre os componentes.
  - **Decisões de projeto discutindo**: alternativas consideradas, opção escolhida e justificativa.
  - **Resultado dos experimentos** comprovando que os requisitos não-funcionais são atendidos.
  - **Modelo de custos** apresentando a estimativa de custo mensal em USD (região us-east-1) para todos os recursos utilizados, com análise comparativa entre o cenário de operação normal e o de evento especial.

Trabalho

Apresentação

- Dias **06/04 e 08/04** — apresentação em sala (ordem definida previamente):
  - O sistema deve estar implantado no momento da apresentação.
  - 20 minutos por grupo + 5 minutos de perguntas.
  - Demonstração funcional: criação de pedido com cálculo de rota, registro de eventos e consulta de histórico.
  - Demonstração de escalabilidade: simulador gerando carga com *auto-scaling* adaptando os recursos dinamicamente.
  - Demonstração de tolerância a falhas: encerrar uma instância de computação ao vivo e mostrar que o serviço se mantém disponível.
  - Todos os integrantes devem ser capazes de responder perguntas sobre qualquer componente do sistema.

Trabalho

Entrega

- Grupos de 3 a 4 integrantes.
- **Entrega:**
  - Submissão no eClass contendo o código-fonte e o relatório em PDF.
  - Prazo: 21/04/2026 às 23:59.
  - Trabalhos entregues após essa data perdem 1 ponto por dia.
  - Data limite (com perda de pontos): 23/04/2026 às 23:59.
- **Formação dos grupos:**
  - Enviar e-mail para **thiago.p.araujo@fgv.br** até 25/03/2026 informando nome completo e e-mail de cada integrante.

Trabalho

Avaliação

- A avaliação será baseada nos critérios a seguir:
  - Qualidade das decisões arquiteturais e justificativas no relatório.
  - Resultados experimentais nos três cenários de carga.
  - Tolerância a falhas demonstrada ao vivo e analisada no relatório.
  - Automação da implantação.
  - Modelo de custos.
  - Corretude dos requisitos funcionais.
  - Organização do projeto e qualidade do código.